SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I S.HARISH/192521003 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S.HARISH

S.HARISH/192521003

Q1 – BACKTRACKING SEARCH : HOSPITAL DOCTOR-SHIFT PROBLEM

Question:

A hospital needs to assign doctors to shifts for one day. There are 3 doctors (Dr. A, Dr. B, Dr. C) and 3 shifts (Morning, Afternoon, Night). Use Backtracking Search to find a valid assignment satisfying all the following constraints:

C1: Each doctor must be assigned exactly one shift.

C2: No two doctors can be assigned the same shift.

C3: Dr. A cannot work the Night shift (personal constraint).

C4: Dr. B cannot work the Morning shift (medical leave in morning).

C5: Dr. C must work either Morning or Afternoon only.

Answer:

Backtracking Search is a systematic method to solve Constraint Satisfaction Problems (CSPs). It assigns values to variables one at a time and checks constraints after each assignment. If a constraint is violated, it backtracks and tries the next possible value. This avoids exploring the full search tree.

Variables and Domains:

Variables are the doctors. Each doctor must be assigned one shift.

Variable	Represents	Domain (possible shifts)
A	Dr. A	Morning, Afternoon, Night
B	Dr. B	Morning, Afternoon, Night
C	Dr. C	Morning, Afternoon, Night

Constraints Summary:

Constraint	Condition
C1	Each doctor gets exactly one shift
C2	All three shifts must be different (all-diff)
C3	A ≠ Night
C4	B ≠ Morning
C5	C ∈ {Morning, Afternoon}

Backtracking Execution – Step by Step:

Order of Assignment: Dr. A → Dr. B → Dr. C

Step 1: Assign Dr. A

Try: A = Morning

• C3 check: A ≠ Night → Morning ≠ Night ✓ PASS

• C2 check: Only A assigned so far, no conflict ✓

Result: A = Morning → ACCEPTED

Step 2: Assign Dr. B (A = Morning)

Try: B = Morning

• C4 check: B ≠ Morning → Morning = Morning ✗ FAIL

Backtrack: B = Morning REJECTED

Try: B = Afternoon

• C4 check: B ≠ Morning → Afternoon ≠ Morning ✓ PASS

• C2 check: A = Morning, B = Afternoon → Morning ≠ Afternoon ✓ PASS

Result: B = Afternoon → ACCEPTED

Step 3: Assign Dr. C (A = Morning, B = Afternoon)

Try: C = Morning

• C5 check: C ∈ {Morning, Afternoon} → Morning ∈ {Morning, Afternoon} ✓ PASS

• C2 check: A = Morning, C = Morning → Morning = Morning ✗ FAIL (duplicate!)

Backtrack: C = Morning REJECTED

Try: C = Afternoon

• C5 check: C ∈ {Morning, Afternoon} → Afternoon ∈ {Morning, Afternoon} ✓ PASS

• C2 check: B = Afternoon, C = Afternoon → Afternoon = Afternoon ✗ FAIL (duplicate!)

Backtrack: C = Afternoon REJECTED

Try: C = Night

• C5 check: C ∈ {Morning, Afternoon} → Night ∉ {Morning, Afternoon} ✗ FAIL

Backtrack: C = Night REJECTED

All values for C exhausted! → Backtrack to Dr. B

Step 4: Re-assign Dr. B (A = Morning, B resumes)

Try: B = Night (next value after Afternoon)

• C4 check: B ≠ Morning → Night ≠ Morning ✓ PASS

• C2 check: A = Morning, B = Night → Morning ≠ Night ✓ PASS

Result: B = Night → ACCEPTED

Step 5: Assign Dr. C (A = Morning, B = Night)

Try: C = Morning

• C5 check: Morning ∈ {Morning, Afternoon} ✓ PASS

• C2 check: A = Morning, C = Morning → DUPLICATE ✗ FAIL

Backtrack: C = Morning REJECTED

Try: C = Afternoon

• C5 check: Afternoon ∈ {Morning, Afternoon} ✓ PASS

• C2 check: A = Morning, B = Night, C = Afternoon → all different ✓ PASS

• C1 check: Dr. C assigned exactly one shift ✓ PASS

Result: C = Afternoon → ACCEPTED

Final Valid Schedule:

Doctor	Assigned Shift
Dr. A	Morning
Dr. B	Night
Dr. C	Afternoon

Final Verification – All 5 Constraints:

Constraint	Check	Result
C1: Each doctor gets exactly one shift	A→Morning, B→Night, C→Afternoon. All assigned exactly once	✓ PASS
C2: No two doctors share the same shift	Morning ≠ Night ≠ Afternoon. All unique	✓ PASS
C3: Dr. A ≠ Night	A = Morning ≠ Night	✓ PASS
C4: Dr. B ≠ Morning	B = Night ≠ Morning	✓ PASS
C5: Dr. C ∈ {Morning, Afternoon}	C = Afternoon ∈ {Morning, Afternoon}	✓ PASS

All 5 constraints are satisfied. The backtracking algorithm found the valid solution after exploring and rejecting 5 invalid assignments.

Q2 – Robot Grid Navigation (5×5 Grid)

Question:

A robot is placed in a 5×5 grid. It starts at position S and must reach goal G. Some cells contain obstacles that the robot cannot enter. Use BFS (Breadth-First Search) to find the path from S to G. Show node expansion, queue updates, Manhattan distance calculation (heuristic), path, and total cost. Verify the result.

Clarification on Terminology (BFS vs Manhattan Heuristic):

The question mentions BFS but also asks for Manhattan distance (which is normally used in A* or Greedy Best-First Search). BFS does not use a heuristic — it explores all nodes level by level and guarantees the shortest path in terms of number of steps.

Grid Setup:

Grid size: 5 × 5. Rows and columns numbered 0 to 4.

Coordinates	Cell Content
(0,0)	S – Start
(2,1)	X – Obstacle
(2,3)	X – Obstacle
(1,3)	X – Obstacle
(4,4)	G – Goal

Grid Layout (text representation): (Row, Col format. X = obstacle)

Col 0 1 2 3 4

Row 0: [S][.][.][.][.]

Row 1: [.][.][.][X][.]

Row 2: [.][X][.][X][.]

Row 3: [.][.][.][.][.]

Row 4: [.][.][.][.][G]

Manhattan Distance Formula:

Manhattan Distance $h(n) = |row_n - row_G| + |col_n - col_G|$

Goal G is at (4,4). So $h(r,c) = |r - 4| + |c - 4|$

BFS Execution – Step by Step:

BFS uses a FIFO queue. Movement allowed: Up, Down, Left, Right (no diagonals). Each step costs 1. Mark visited nodes to avoid re-expansion.

Initial State:

Queue: [(0,0)]

Visited: { (0,0) }

Parent map: { (0,0): None }

Step 1 – Expand (0,0) [Start = S]

h(0,0) = |0−4| + |0−4| = 4 + 4 = 8 (Manhattan distance for reference)

Neighbors: Down→(1,0), Right→(0,1) [Up and Left are out of bounds]

Add to queue: (1,0), (0,1)

Queue: [(1,0), (0,1)]

Visited: { (0,0), (1,0), (0,1) }

Step 2 – Expand (1,0)

h(1,0) = |1−4| + |0−4| = 3 + 4 = 7

Neighbors: Up→(0,0) [visited], Down→(2,0), Right→(1,1), Left→out of bounds

Add to queue: (2,0), (1,1)

Queue: [(0,1), (2,0), (1,1)]

Visited: { (0,0), (1,0), (0,1), (2,0), (1,1) }

Step 3 – Expand (0,1)

h(0,1) = |0−4| + |1−4| = 4 + 3 = 7

Neighbors: Left→(0,0) [visited], Right→(0,2), Down→(1,1) [visited]

Add to queue: (0,2)

Queue: [(2,0), (1,1), (0,2)]

Visited adds: (0,2)

Step 4 – Expand (2,0)

h(2,0) = |2−4| + |0−4| = 2 + 4 = 6

Neighbors: Up→(1,0) [visited], Down→(3,0), Right→(2,1) [X obstacle – SKIP]

Add to queue: (3,0)

Queue: [(1,1), (0,2), (3,0)]

Visited adds: (3,0)

Step 5 – Expand (1,1)

h(1,1) = |1−4| + |1−4| = 3 + 3 = 6

Neighbors: Up→(0,1) [visited], Down→(2,1) [obstacle – SKIP], Left→(1,0) [visited], Right→(1,2)

Add to queue: (1,2)

Queue: [(0,2), (3,0), (1,2)]

Visited adds: (1,2)

Step 6 – Expand (0,2)

h(0,2) = |0−4| + |2−4| = 4 + 2 = 6

Neighbors: Left→(0,1) [visited], Right→(0,3), Down→(1,2) [visited]

Add to queue: (0,3)

Queue: [(3,0), (1,2), (0,3)]

Visited adds: (0,3)

Step 7 – Expand (3,0)

h(3,0) = |3−4| + |0−4| = 1 + 4 = 5

Neighbors: Up→(2,0) [visited], Down→(4,0), Right→(3,1)

Add to queue: (4,0), (3,1)

Queue: [(1,2), (0,3), (4,0), (3,1)]

Visited adds: (4,0), (3,1)

Step 8 – Expand (1,2)

h(1,2) = |1−4| + |2−4| = 3 + 2 = 5

Neighbors: Up→(0,2) [visited], Down→(2,2), Left→(1,1) [visited], Right→(1,3) [obstacle – SKIP]

Add to queue: (2,2)

Queue: [(0,3), (4,0), (3,1), (2,2)]

Visited adds: (2,2)

Step 9 – Expand (0,3)

h(0,3) = |0−4| + |3−4| = 4 + 1 = 5

Neighbors: Left→(0,2) [visited], Right→(0,4), Down→(1,3) [obstacle – SKIP]

Add to queue: (0,4)

Queue: [(4,0), (3,1), (2,2), (0,4)]

Visited adds: (0,4)

Step 10 – Expand (4,0)

h(4,0) = |4−4| + |0−4| = 0 + 4 = 4

Neighbors: Up→(3,0) [visited], Right→(4,1)

Add to queue: (4,1)

Queue: [(3,1), (2,2), (0,4), (4,1)]

Visited adds: (4,1)

Step 11 – Expand (3,1)

h(3,1) = |3−4| + |1−4| = 1 + 3 = 4

Neighbors: Up→(2,1) [obstacle – SKIP], Down→(4,1) [visited], Left→(3,0) [visited], Right→(3,2)

Add to queue: (3,2)

Queue: [(2,2), (0,4), (4,1), (3,2)]

Visited adds: (3,2)

Step 12 – Expand (2,2)

h(2,2) = |2−4| + |2−4| = 2 + 2 = 4

Neighbors: Up→(1,2) [visited], Down→(3,2) [visited], Left→(2,1) [obstacle], Right→(2,3) [obstacle]

No new nodes to add.

Queue: [(0,4), (4,1), (3,2)]

Step 13 – Expand (0,4)

h(0,4) = |0−4| + |4−4| = 4 + 0 = 4

Neighbors: Left→(0,3) [visited], Down→(1,4)

Add to queue: (1,4)

Queue: [(4,1), (3,2), (1,4)]

Visited adds: (1,4)

Step 14 – Expand (4,1)

h(4,1) = |4−4| + |1−4| = 0 + 3 = 3

Neighbors: Left→(4,0) [visited], Right→(4,2), Up→(3,1) [visited]

Add to queue: (4,2)

Queue: [(3,2), (1,4), (4,2)]

Visited adds: (4,2)

Step 15 – Expand (3,2)

h(3,2) = |3−4| + |2−4| = 1 + 2 = 3

Neighbors: Up→(2,2) [visited], Down→(4,2) [visited], Left→(3,1) [visited], Right→(3,3)

Add to queue: (3,3)

Queue: [(1,4), (4,2), (3,3)]

Visited adds: (3,3)

Step 16 – Expand (1,4)

h(1,4) = |1−4| + |4−4| = 3 + 0 = 3

Neighbors: Up→(0,4) [visited], Down→(2,4), Left→(1,3) [obstacle]

Add to queue: (2,4)

Queue: [(4,2), (3,3), (2,4)]

Visited adds: (2,4)

Step 17 – Expand (4,2)

h(4,2) = |4−4| + |2−4| = 0 + 2 = 2

Neighbors: Left→(4,1) [visited], Right→(4,3), Up→(3,2) [visited]

Add to queue: (4,3)

Queue: [(3,3), (2,4), (4,3)]

Visited adds: (4,3)

Step 18 – Expand (3,3)

h(3,3) = |3−4| + |3−4| = 1 + 1 = 2

Neighbors: Up→(2,3) [obstacle], Down→(4,3) [visited], Left→(3,2) [visited], Right→(3,4)

Add to queue: (3,4)

Queue: [(2,4), (4,3), (3,4)]

Visited adds: (3,4)

Step 19 – Expand (2,4)

h(2,4) = |2−4| + |4−4| = 2 + 0 = 2

Neighbors: Up→(1,4) [visited], Down→(3,4) [visited], Left→(2,3) [obstacle]

No new nodes.

Queue: [(4,3), (3,4)]

Step 20 – Expand (4,3)

h(4,3) = |4−4| + |3−4| = 0 + 1 = 1

Neighbors: Left→(4,2) [visited], Right→(4,4), Up→(3,3) [visited]

(4,4) = G → GOAL FOUND!

Add (4,4) to queue, stop BFS.

Path Reconstruction (Trace back from G to S using parent map):

(4,4) ← (4,3) ← (4,2) ← (4,1) ← (4,0) ← (3,0) ← (2,0) ← (1,0) ← (0,0)

Final Path: (0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)

Total Cost Calculation:

Each move costs 1 step.

Number of moves:

(0,0)→(1,0): 1 step

(1,0)→(2,0): 1 step

(2,0)→(3,0): 1 step

(3,0)→(4,0): 1 step

(4,0)→(4,1): 1 step

(4,1)→(4,2): 1 step

(4,2)→(4,3): 1 step

(4,3)→(4,4): 1 step

Total cost = 8 steps (8 moves)

Final Verification:

Check	Result
Start node	(0,0) = S ✓
Goal node	(4,4) = G ✓
Obstacle at (2,1) avoided	Path goes through (2,0), not (2,1) ✓
Obstacle at (1,3) avoided	Path does not pass through (1,3) ✓
Obstacle at (2,3) avoided	Path does not pass through (2,3) ✓
All moves valid (adjacent cells)	Each step moves exactly one cell horizontally or vertically ✓
Total cost	8 steps ✓

BFS guarantees this is the shortest path in terms of number of moves.

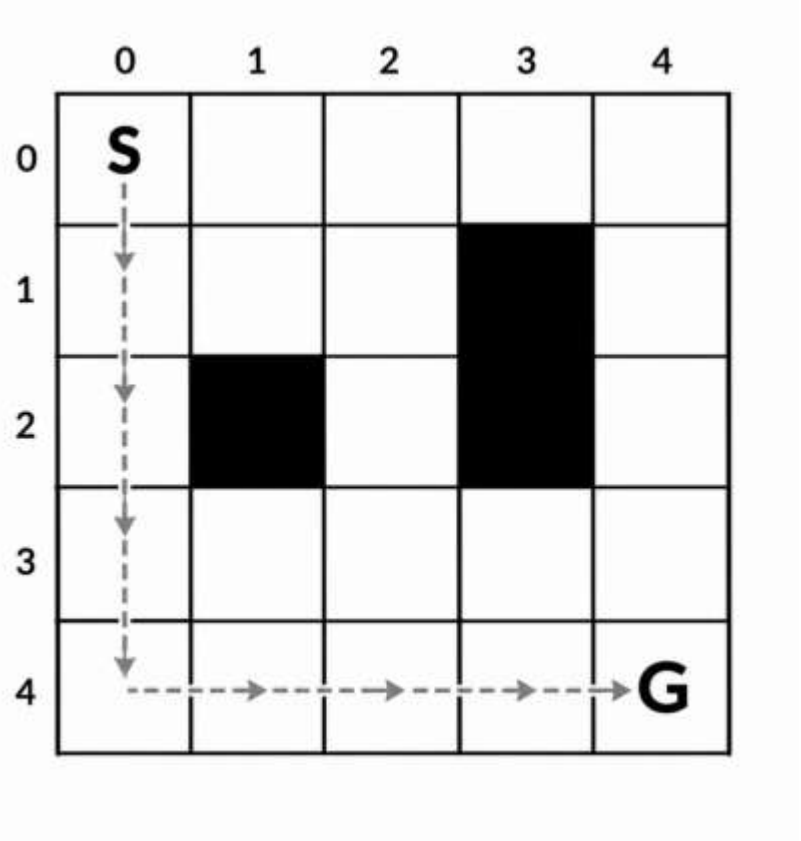
Python Code for Grid Visualization (Google Colab):

Run this in Google Colab to produce a black-and-white PNG of the grid:

```
import matplotlib.pyplot as plt
import matplotlib.patches as patches

grid = [
  [0,0,0,0,0],  # row 0  (0=free, 2=obstacle, 3=path, 4=start, 5=goal)
  [0,0,0,2,0],  # row 1
  [0,2,0,2,0],  # row 2
  [0,0,0,0,0],  # row 3
  [0,0,0,0,0],  # row 4
]
path = [(0,0),(1,0),(2,0),(3,0),(4,0),(4,1),(4,2),(4,3),(4,4)]
for (r,c) in path: grid[r][c] = 3
grid[0][0] = 4  # start
grid[4][4] = 5  # goal
colors = {0:'white', 2:'black', 3:'lightgray', 4:'white', 5:'white'}
fig, ax = plt.subplots(figsize=(5,5))
```

```
for r in range(5):
    for c in range(5):
        fc = colors[grid[r][c]]
        rect = patches.Rectangle((c,4-r),1,1,linewidth=1,edgecolor='black',facecolor=fc)
        ax.add_patch(rect)
        label = ''
        if grid[r][c]==4: label='S'
        elif grid[r][c]==5: label='G'
        elif grid[r][c]==2: label='X'
        elif grid[r][c]==3: label='*'
        ax.text(c+0.5,4-r+0.5,label,ha='center',va='center',fontsize=14,fontweight='bold')
ax.set_xlim(0,5); ax.set_ylim(0,5)
ax.set_aspect('equal'); ax.axis('off')
plt.title('5x5 Robot Grid - BFS Path',fontsize=12)
plt.tight_layout()
plt.savefig('robot_grid.png',dpi=150,bbox_inches='tight',facecolor='white')
plt.show()
```



Q3 – RESCUE ROBOT IN A DISASTER ZONE

Question:

A rescue robot is operating in a disaster zone modeled as a grid. The robot must navigate from a start location to reach survivors while avoiding debris. Some cells are tagged as 'risky zones' (unstable floor). Constraints apply:

- Normal movement cost = 1
- Entering a risky-zone cell has an additional cost of 2, so total cost = 1 + 2 = 3
- The robot must not revisit the same cell (loop avoidance)
- The robot must reach the survivor in minimum total cost

(a) What is the effect of each constraint on the robot's navigation?

(b) Which search strategy is most suitable? Explain with steps.

(c) What type of intelligent agent is the rescue robot? Justify.

(d) Describe the environment type for this problem. Is the robot rational?

Answer:

(a) Effect of Each Constraint on Robot Navigation:

Constraint 1: Normal movement cost = 1

Every move the robot makes from one cell to an adjacent non-risky cell costs 1 unit. This uniform base cost allows the search algorithm to count the number of moves taken and compare path costs accurately. Without this, there would be no way to measure which path is cheaper.

Constraint 2: Risky-zone entry cost = 1 + 2 = 3

When the robot enters a risky-zone cell, it incurs a normal movement cost of 1 PLUS an additional penalty of 2 for the risk. So the total cost for that one move becomes 3.

Calculation: Move cost = 1 (base) + 2 (risky penalty) = 3

Effect: The robot will avoid risky cells whenever a safer path with lower total cost exists. This naturally steers the robot away from dangerous areas. However, if the only path to the survivor goes through a risky zone, the robot will take it but at a higher cost.

Constraint 3: No revisiting cells (loop avoidance)

The robot marks each visited cell and does not enter it again. Effect: This prevents the robot from going in circles and wasting cost. It also ensures the search terminates. Without this constraint, the robot could loop forever between two cells.

Constraint 4: Minimum cost path to survivor

The robot's goal is not just to reach the survivor, but to do so with the lowest total cost. This means the search algorithm must explore all possible paths and select the cheapest one, not just the first one found.

(B) SUITABLE SEARCH STRATEGY – UNIFORM COST SEARCH (UCS):

UCS (Uniform Cost Search) expands the node with the lowest cumulative cost from the start. It is essentially Dijkstra's algorithm for tree search.

Example Grid for UCS demonstration:

Suppose the grid is 4×4. Start S at (0,0), Survivor G at (3,3). Risky cells: (1,1) and (2,2).

```
     Col  0    1    2    3
Row 0:  [  S  ][  .  ][  .  ][  .  ]
Row 1:  [  .  ][  R  ][  .  ][  .  ]    R = Risky zone
Row 2:  [  .  ][  .  ][  R  ][  .  ]
Row 3:  [  .  ][  .  ][  .  ][  G  ]
```

UCS Step-by-Step Operation:

UCS uses a Priority Queue ordered by cumulative cost g(n).

Initial State:

Priority Queue: [(0, (0,0))] — (cost, node)

Visited: {}

Step 1 – Expand (0,0), cost=0

Neighbors: (0,1) cost=1, (1,0) cost=1

Priority Queue: [(1,(0,1)), (1,(1,0))]

Visited: {(0,0)}

Step 2 – Expand (0,1), cost=1 [tie-break by position]

Neighbors: (0,0) visited, (0,2) cost=1+1=2, (1,1) is RISKY cost=1+3=4

Priority Queue: [(1,(1,0)), (2,(0,2)), (4,(1,1))]

Visited: {(0,0), (0,1)}

Step 3 – Expand (1,0), cost=1

Neighbors: (0,0) visited, (2,0) cost=1+1=2, (1,1) RISKY cost=1+3=4

Priority Queue: [(2,(0,2)), (2,(2,0)), (4,(1,1)), (4,(1,1))]

Note: (1,1) added twice from different paths — only the lower cost entry is used when expanded.

Visited: {(0,0),(0,1),(1,0)}

Step 4 – Expand (0,2), cost=2

Neighbors: (0,1) visited, (0,3) cost=3, (1,2) cost=3

Priority Queue: [(2,(2,0)), (3,(0,3)), (3,(1,2)), (4,(1,1)), (4,(1,1))]

Visited adds: (0,2)

Step 5 – Expand (2,0), cost=2

Neighbors: (1,0) visited, (3,0) cost=3, (2,1) cost=3

Priority Queue: [(3,(0,3)), (3,(1,2)), (3,(3,0)), (3,(2,1)), (4,(1,1)), (4,(1,1))]

Visited adds: (2,0)

Step 6 – Expand (0,3), cost=3

Neighbors: (0,2) visited, (1,3) cost=4

Priority Queue: [(3,(1,2)), (3,(3,0)), (3,(2,1)), (4,(1,1)), (4,(1,1)), (4,(1,3))]

Visited adds: (0,3)

Step 7 – Expand (1,2), cost=3

Neighbors: (0,2) visited, (1,3) cost=4, (2,2) RISKY cost=3+3=6, (1,1) RISKY cost=3+3=6

Priority Queue: [(3,(3,0)), (3,(2,1)), (4,(1,1)), (4,(1,1)), (4,(1,3)), (4,(1,3)), (6,(2,2)), (6,(1,1))]

Visited adds: (1,2)

Step 8 – Expand (3,0), cost=3

Neighbors: (2,0) visited, (3,1) cost=4

Visited adds: (3,0)

Step 9 – Expand (2,1), cost=3

Neighbors: (2,0) visited, (1,1) RISKY cost=3+3=6, (3,1) cost=4, (2,2) RISKY cost=3+3=6

Visited adds: (2,1)

Steps 10–14: Continue expanding lower-cost nodes...

Expanding (1,3) cost=4, (3,1) cost=4 → add (3,2) cost=5, (2,3) cost=5

Expanding (3,2) cost=5 → add (3,3) cost=6

Expanding (2,3) cost=5 → add (3,3) at cost=6

Goal Check: Expand (3,3), cost=6 → GOAL FOUND!

Optimal Path (UCS):

(0,0) → (0,1) → (0,2) → (1,2) → (1,3) → (2,3) → (3,3)

Cost Calculation:

(0,0)→(0,1) = 1

(0,1)→(0,2) = 1 → cumulative = 2

(0,2)→(1,2) = 1 → cumulative = 3

(1,2)→(1,3) = 1 → cumulative = 4

(1,3)→(2,3) = 1 → cumulative = 5

(2,3)→(3,3) = 1 → cumulative = 6

Total Minimum Cost = 6 (no risky zones entered on this path)

Comparison: If the robot went through (1,1) which is risky, cost would be higher (penalty of 2 extra per risky cell). UCS correctly avoids them.

(c) Type of Intelligent Agent:

The rescue robot is a Goal-Based Agent with properties of a Utility-Based Agent.

Goal-Based Agent:

The robot has a clear goal: reach the survivor location. It uses search algorithms to plan a sequence of actions to achieve this goal. Every decision (which cell to move to next) is made with the goal in mind.

Utility-Based Agent aspect:

The robot also optimizes cost. It does not just reach the goal, it reaches it with minimum total cost. The cost function acts as a utility measure — lower cost = higher utility. So the robot is technically a utility-maximizing (or cost-minimizing) goal-based agent.

Justification:

A reflex agent reacts to current input only — not suitable here because the robot needs to plan ahead through multiple cells.

A model-based agent maintains an internal model of the world — the robot does this by keeping the grid map and marking visited cells.

A goal-based agent uses this model to plan paths — the robot uses UCS to find the optimal path to the goal.

So the rescue robot is best described as a Model-Based, Goal-Based, Utility-Driven Intelligent Agent.

(d) Environment Type and Rationality:

Environment Type:

Property	Value	Reason
Observable	Fully Observable	Robot can see the entire grid including risky zones
Deterministic	Deterministic	Outcome of each move is known and fixed
Episodic vs Sequential	Sequential	Earlier moves affect later positions and available paths
Static vs Dynamic	Static	Grid does not change while robot is navigating

Discrete	Discrete	Grid cells are discrete positions
Single vs Multi Agent	Single Agent	Only one robot is navigating

Is the robot rational?

Yes, the rescue robot is rational.

A rational agent is one that selects the action that maximizes its performance measure given its knowledge. The robot's performance measure here is reaching the survivor with minimum cost.

B)– UNIFORM COST SEARCH / A* ON A WEIGHTED GRAPH

Question:

Given the following weighted undirected graph, apply Uniform Cost Search (UCS) to find the minimum cost path from node S to node G. Show the priority queue at each step, cumulative costs, node expansions, and the final path with cost.

Graph edges (node – node : cost):

S – A : 2

S – B : 4

A – C : 3

A – B : 1

B – C : 2

B – D : 5

C – G : 4

D – G : 2

Answer:

Uniform Cost Search (UCS) expands the node with the lowest cumulative path cost g(n) from the start. It uses a priority queue (min-heap) ordered by g(n). UCS is complete and optimal when all edge costs are non-negative.

Graph Summary:

From	To	Cost
S	A	2
S	B	4
A	C	3
A	B	1
B	C	2
B	D	5
C	G	4
D	G	2

UCS Execution:

Priority Queue format: (g-cost, node, path)

Initial State:

Priority Queue: [(0, S, [S])]

Visited: {}

Step 1 – Pop (0, S)

Expand S. Mark S as visited.

Neighbors:

A: g = 0 + 2 = 2 → add (2, A, [S,A])

B: g = 0 + 4 = 4 → add (4, B, [S,B])

Priority Queue: [(2, A, [S,A]), (4, B, [S,B])]

Visited: { S }

Step 2 – Pop (2, A)

Expand A. Mark A as visited.

Neighbors:

S: already visited – SKIP

C: g = 2 + 3 = 5 → add (5, C, [S,A,C])

B: g = 2 + 1 = 3 → add (3, B, [S,A,B])

Priority Queue: [(3, B, [S,A,B]), (4, B, [S,B]), (5, C, [S,A,C])]

Visited: { S, A }

Step 3 – Pop (3, B, [S,A,B])

Expand B. Mark B as visited.

Neighbors:

S: visited – SKIP

A: visited – SKIP

C: g = 3 + 2 = 5 → add (5, C, [S,A,B,C])

D: g = 3 + 5 = 8 → add (8, D, [S,A,B,D])

Priority Queue: [(4, B, [S,B]), (5, C, [S,A,C]), (5, C, [S,A,B,C]), (8, D, [S,A,B,D])]

Visited: { S, A, B }

Step 4 – Pop (4, B, [S,B])

B is already VISITED – SKIP this entry.

Priority Queue: [(5, C, [S,A,C]), (5, C, [S,A,B,C]), (8, D, [S,A,B,D])]

Step 5 – Pop (5, C, [S,A,C]) [tie-break: first in queue]

Expand C. Mark C as visited.

Neighbors:

A: visited – SKIP

B: visited – SKIP

G: g = 5 + 4 = 9 → add (9, G, [S,A,C,G])

Priority Queue: [(5, C, [S,A,B,C]), (8, D, [S,A,B,D]), (9, G, [S,A,C,G])]

Visited: { S, A, B, C }

Step 6 – Pop (5, C, [S,A,B,C])

C is already VISITED – SKIP.

Priority Queue: [(8, D, [S,A,B,D]), (9, G, [S,A,C,G])]

Step 7 – Pop (8, D, [S,A,B,D])

Expand D. Mark D as visited.

Neighbors:

B: visited – SKIP

G: g = 8 + 2 = 10 → add (10, G, [S,A,B,D,G])

Priority Queue: [(9, G, [S,A,C,G]), (10, G, [S,A,B,D,G])]

Visited: { S, A, B, C, D }

Step 8 – Pop (9, G, [S,A,C,G])

G is the GOAL! → SEARCH COMPLETE.

Final Result:

Optimal Path: S → A → C → G

Cost Breakdown:

S → A : 2

A → C : 3 → cumulative = 5

C → G : 4 → cumulative = 9

Total Minimum Cost = 9

All Paths Compared:

Path	Cost	Selected?
S→A→C→G	2+3+4 = 9	YES – OPTIMAL
S→A→B→C→G	2+1+2+4 = 9	Same cost, not selected (same)
S→B→C→G	4+2+4 = 10	No
S→A→B→D→G	2+1+5+2 = 10	No
S→B→D→G	4+5+2 = 11	No

Final Verification:

Path: S → A → C → G

S–A edge exists: cost 2 ✓

A–C edge exists: cost 3 ✓

C–G edge exists: cost 4 ✓

Total: 2 + 3 + 4 = 9 ✓

UCS guarantees optimality — no other path has a lower cost than 9. ✓

A* Search on a Weighted Graph with Heuristic

C) QUESTION:

APPLY A* SEARCH ON THE SAME GRAPH AS Q4. USE THE FOLLOWING STRAIGHT-LINE DISTANCE HEURISTIC VALUES H(N) FOR EACH NODE. SHOW THE OPEN LIST (PRIORITY QUEUE), F(N) = G(N) + H(N) AT EACH STEP, NODE EXPANSIONS, AND THE FINAL OPTIMAL PATH.

Heuristic values h(n) (estimated cost to Goal G):

h(S) = 7, h(A) = 5, h(B) = 6, h(C) = 4, h(D) = 2, h(G) = 0

Graph edges (same as Q4):

S–A:2, S–B:4, A–C:3, A–B:1, B–C:2, B–D:5, C–G:4, D–G:2

Answer:

A* Search combines the actual cost g(n) (cost from start to n) with the heuristic h(n) (estimated cost from n to goal) to compute f(n) = g(n) + h(n). The node with the lowest f(n) is expanded first. A* is optimal and complete when the heuristic is admissible (never overestimates the true cost).

Admissibility check: True path cost from each node to G — we verify h(n) ≤ true cost.

h(S)=7, true cost S to G = 9. 7 ≤ 9 ✓

h(A)=5, true cost A to G = 7 (A→C→G). 5 ≤ 7 ✓

h(B)=6, true cost B to G = 6 (B→C→G). 6 ≤ 6 ✓ (equals, still admissible)

h(C)=4, true cost C to G = 4. 4 ≤ 4 ✓

h(D)=2, true cost D to G = 2. 2 ≤ 2 ✓

h(G)=0 ✓

All heuristic values are admissible. A* will find the optimal solution.

A* Execution – Step by Step:

Open List format: (f, g, node, path)

Initial State:

Open List: [(0+7=7, 0, S, [S])]

Closed Set: {}

Step 1 – Pop (f=7, g=0, S)

Expand S. Add S to Closed Set.

Neighbors:

A: g=0+2=2, f=2+h(A)=2+5=7 → add (7, 2, A, [S,A])

B: g=0+4=4, f=4+h(B)=4+6=10 → add (10, 4, B, [S,B])

Open List: [(7, 2, A, [S,A]), (10, 4, B, [S,B])]

Closed: { S }

Step 2 – Pop (f=7, g=2, A)

Expand A. Add A to Closed Set.

Neighbors:

S: in Closed – SKIP

C: g=2+3=5, f=5+h(C)=5+4=9 → add (9, 5, C, [S,A,C])

B: g=2+1=3, f=3+h(B)=3+6=9 → add (9, 3, B, [S,A,B])

Open List: [(9, 5, C, [S,A,C]), (9, 3, B, [S,A,B]), (10, 4, B, [S,B])]

Closed: { S, A }

Step 3 – Pop (f=9, g=5, C, [S,A,C]) [tie: lower f first; tie-break by g: C has g=5 vs B g=3; actually both f=9, so tie-break by index – pop C first as it was added first]

Expand C. Add C to Closed Set.

Neighbors:

A: in Closed – SKIP

B: not in Closed. g=5+2=7, f=7+h(B)=7+6=13. Compare with existing B entries:

(9,3,B) and (10,4,B) are already in open with better f — do NOT replace.

G: g=5+4=9, f=9+h(G)=9+0=9 → add (9, 9, G, [S,A,C,G])

Open List: [(9, 3, B, [S,A,B]), (9, 9, G, [S,A,C,G]), (10, 4, B, [S,B])]

Closed: { S, A, C }

Step 4 – Pop (f=9, g=3, B, [S,A,B]) [tie: g=3 vs g=9 for G; lower g means better actual path, and in A* ties in f are broken by lower g or FIFO — B popped first]

Expand B. Add B to Closed Set.

Neighbors:

S: in Closed – SKIP

A: in Closed – SKIP

C: in Closed – SKIP

D: g=3+5=8, f=8+h(D)=8+2=10 → add (10, 8, D, [S,A,B,D])

Open List: [(9, 9, G, [S,A,C,G]), (10, 4, B, [S,B]), (10, 8, D, [S,A,B,D])]

Closed: { S, A, C, B }

Step 5 – Pop (f=9, g=9, G, [S,A,C,G])

G is the GOAL! → SEARCH COMPLETE.

D) JUSTIFY WHY YOUR CHOSEN APPROACH IS THE MOST SUITABLE FOR SOLVING THIS PROBLEM UNDER THE GIVEN CONSTRAINTS.

Optimal Path: S → A → C → G

Cost Breakdown:

g(S) = 0

g(A) = 0 + 2 = 2

g(C) = 2 + 3 = 5

g(G) = 5 + 4 = 9

Total Minimum Cost = 9

f(n) Values at Each Expansion:

Step	Node Expanded	g(n)	h(n)	f(n)
1	S	0	7	7
2	A	2	5	7
3	C	5	4	9
4	B	3	6	9
5	G (Goal)	9	0	9

Comparison: A* vs UCS on This Problem:

Property	UCS (Q4)	A* (Q5)
Algorithm	Expand lowest g(n)	Expand lowest f(n)=g(n)+h(n)
Nodes expanded	More (no heuristic guidance)	Fewer (heuristic guides search)
Optimal?	Yes	Yes (with admissible h)
Final path	S→A→C→G	S→A→C→G
Final cost	9	9

Both algorithms find the same optimal path with cost 9. A* is faster because the heuristic avoids expanding less-promising nodes.

Final Verification:

Path: S → A → C → G

S–A edge: 2 ✓

A–C edge: 3 ✓

C–G edge: 4 ✓

Total: 2 + 3 + 4 = 9 ✓

h values are admissible (never overestimate) ✓